Bit Manipulation & Bitmask Enumeration

An integer is a set of up to 64 booleans

Learning goals

- Read and write numbers in **binary**; know two's complement
- Use `& | ^ ~ << >>` fluently, and the operator-precedence traps
- Classic tricks: test / set / clear / toggle a bit, **lowest set bit**
- GCC builtins: `popcount`, `ctz`, `clz`, and `std::bitset`
- Represent a subset as a **bitmask**; enumerate all 2^n subsets
- Enumerate **submasks** in $O(3^n)$ total
- Use **XOR** properties to solve problems in one pass

Motivation: CSES 1623 — Apple Division

$n \le 20$ apples with weights p_i. Split them into two groups so that the difference of the group weights is minimal.

```
5
3 2 7 4 1      →  answer 1   ({3,2,4} = 9 vs {7,1} = 8)
```

Last week: recursion over 2^n include/exclude choices.

This week: **every integer `mask` in $[0, 2^n)$ *is* a subset.**

One `for` loop, no recursion.

Binary refresher

```
44 = 32 + 8 + 4 = 101100₂

bit index:  5 4 3 2 1 0
bit value:  1 0 1 1 0 0
```

- Bit *i* has value 2^i; bit 0 is the **least significant** bit
- `int` = 32 bits, `long long` = 64 bits
- **Two's complement**: `-x == ~x + 1`
  `-1` is all ones; `INT_MIN` is `1000…0`
- Prefer **unsigned** or non-negative values when shifting

The operators

Op	Name	Example (12 = 1100, 10 = 1010)
`a & b`	AND	`1000` = 8
`a | b`	OR	`1110` = 14
`a ^ b`	XOR	`0110` = 6
`~a`	NOT	flips all 32 bits
`a << k`	left shift	$a \cdot 2^k$
`a >> k`	right shift	$\lfloor a / 2^k \rfloor$ for $a \geq 0$

⚠️ Precedence: `x & 1 == 0` parses as `x & (1 == 0)`. **Use parentheses.**

Single-bit operations

```
bool on  = (x >> i) & 1;   // test bit i
x |=  (1 << i);            // set bit i
x &= ~(1 << i);            // clear bit i
x ^=  (1 << i);            // toggle bit i
```

For 64-bit: `1LL << i`. Writing `1 << 40` is undefined behaviour (an `int` shifted past its width), not 2^{40}.

Lowest set bit tricks

```
x           = 0010 1100
x - 1       = 0010 1011    (borrow flips the trailing 100 → 011)
x & (x-1)   = 0010 1000    clear lowest set bit
-x          = 1101 0100
x & -x      = 0000 0100    isolate lowest set bit
```

- `x & (x - 1) == 0` ⇔ x is 0 or a **power of two**
- `x & -x` is the heart of the Fenwick tree (Week 24)
- Loop over set bits: `for (t = x; t; t &= t - 1) use(__builtin_ctz(t));`

GCC builtins

Call	Meaning	44 = 101100
`__builtin_popcount(x)`	number of 1 bits	3
`__builtin_ctz(x)`	trailing zeros	2
`__builtin_clz(x)`	leading zeros (32-bit)	26
`31 - __builtin_clz(x)`	index of highest bit = $\lfloor \log_2 x \rfloor$	5

- 64-bit versions: `__builtin_popcountll`, `ctzll`, `clzll`
- `ctz(0)` and `clz(0)` are **undefined** — guard against 0
- C++20 has `std::popcount`, `std::countr_zero`, `std::bit_width`

`std::bitset`

```
bitset<8> b(44);              // 00101100
b[0] = 1;                     // 00101101
cout << b.count();            // 4
cout << b;                    // prints 00101101
bitset<100000> A, B;
auto C = A & B;               // n/64 word operations
```

- Fixed size at compile time
- Bulk operations are ~64× faster than `bool` arrays
- Useful for subset-sum style DP: `reach |= reach << w`

Subsets as bitmasks

Universe {0, 1, …, n−1}. A set S ↔ integer with bit *i* set iff i ∈ S.

Set operation	Bitmask
∅ / full set	`0` / `(1 << n) - 1`
i ∈ S	`S >> i & 1`
S ∪ T, S ∩ T	`S | T`, `S & T`
S \ T	`S & ~T`
complement	`full ^ S`
|S|	`__builtin_popcount(S)`
S ⊆ T	`(S & T) == S`

Enumerating all subsets

```
for (int mask = 0; mask < (1 << n); mask++) {
    long long s = 0;
    for (int i = 0; i < n; i++)
        if (mask >> i & 1) s += p[i];
    best = min(best, llabs(total - 2 * s));
}
```

- That's the full Apple Division solution: **$O(n \cdot 2^n)$**
- n = 20: $2 \cdot 10^7$ operations — instant
- Masks go in increasing order: every proper submask of `mask` comes **before** it (used later in bitmask DP)

Incremental subset sums: $O(2^n)$

Reuse the sum of `mask` without its lowest bit:

```
vector<long long> sum(1 << n, 0);
for (int mask = 1; mask < (1 << n); mask++) {
    int i = __builtin_ctz(mask);              // lowest element
    sum[mask] = sum[mask & (mask - 1)] + p[i];
}
```

Your first taste of **DP over subsets** (Week 26).

Memory: $2^{20} \cdot 8$ bytes = 8 MB — check limits.

Enumerating submasks of a mask

```
for (int s = m; s > 0; s = (s - 1) & m) {
    // s is a non-empty submask of m, decreasing order
}
// handle s = 0 separately if needed
```

m = 1011 → `1011 1010 1001 1000 0011 0010 0001`

Why: `s - 1` flips the lowest 1 and all zeros below it;
`& m` drops bits outside m → the next smaller submask.

All submasks of all masks: $O(3^n)$

```
for (int m = 0; m < (1 << n); m++)
    for (int s = m; ; s = (s - 1) & m) {
        work(m, s);
        if (s == 0) break;
    }
```

Each element is in one of 3 states: not in m / in m but not s / in s.

Total pairs = $\mathbf{3^n}$, not 4^n.

n = 15: $1.4 \cdot 10^7$ ✓; n = 20: $3.5 \cdot 10^9$ ×.

(Verified: n = 10 gives exactly 59,049 = 3^{10} iterations.)

XOR: the magic operator

- `a ^ a = 0`, `a ^ 0 = a`, commutative & associative
- So XOR of a list **cancels pairs**:
  find the one element that appears an odd number of times
- `a ^ b` has bit *i* set ⇔ a and b differ at bit *i*
- Prefix XOR: `px[r] ^ px[l-1]` = XOR of a[l..r] (like Week 7 prefix sums)
- Swap without temp: `a ^= b; b ^= a; a ^= b;` (cute, never needed)
- `a + b == (a ^ b) + 2 * (a & b)` — sum = carry-less sum + carries

Thinking bit by bit

Many problems with AND/OR/XOR are solved **independently per bit**:

> Sum of `a[i] ^ a[j]` over all pairs $i < j$, $n = 10^5$

For each bit b: let c_1 = count of numbers with bit b set, $c_0 = n - c_1$.
Pairs that differ at bit b: $c_0 \cdot c_1$. Contribution: $c_0 \cdot c_1 \cdot 2^b$.

O(n · log maxA) instead of $O(n^2)$.

Gray code (CSES 2205)

List all 2^n bit strings so that neighbours differ in exactly one bit.

```
for (int i = 0; i < (1 << n); i++) {
    int g = i ^ (i >> 1);       // i-th Gray code
    print(g, n);
}
```

n = 3: `000 001 011 010 110 111 101 100`

Use: iterate over subsets changing **one element at a time**

→ O(1) update per subset instead of O(n).

Bitmasks as compact state

- Visited set of ≤ 20 cities → one `int` (Week 26: TSP DP)
- Which letters occur in a word → 26-bit mask;
  two words share no letter ⇔ `(m1 & m2) == 0`
- Grid row of ≤ 20 cells → one int; neighbouring cells:
  `row << 1`, `row >> 1`
- Parity of counts of each digit → 10-bit mask
  ("is a substring rearrangeable into a palindrome?")

Pitfalls

- `1 << 31` overflows `int` → use `1u << 31` or `1LL << 31`
- `1 << n` with n ≥ 32 on `int`, ≥ 64 on `long long`: **undefined**
- Precedence: `==`, `<` bind tighter than `&`, `^`, `|`
- `__builtin_popcount` on a `long long` silently truncates → use `...ll`
- `ctz(0)` / `clz(0)` undefined
- Right-shifting **negative** numbers: implementation-defined before C++20
- 2^n memory: arrays of size `1 << 25` ints = 128 MB

Complexity cheat sheet

Task	Time
test/set/clear/toggle, `x & -x`, builtins	$O(1)$
iterate set bits of x	$O(\text{popcount})$
all subsets, check each in $O(n)$	$O(n \cdot 2^n)$
all subsets with incremental update	$O(2^n)$
all (mask, submask) pairs	$O(3^n)$
bitset of size N, bulk op	$O(N / 64)$

Practice problems

- CSES 1623 — Apple Division
- CSES 2205 — Gray Code
- CSES 1146 — Counting Bits (count 1 bits in 1..n, $n \le 10^{15}$)
- Codeforces 579A — Raising Bacteria
- Codeforces 1097B — Petr and a Combination Lock
- Codeforces 550B — Preparing Olympiad (bitmask version)
- AtCoder ABC 147 C — HonestOrUnkind2
- Codeforces 1420B — Rock and Lever
- Codeforces 1615B — And It's Non-Zero

Summary

- An integer is a set: `&` ∩, `|` ∪, `^` symmetric difference
- Know the one-liners: test/set/clear/toggle, `x & (x-1)`, `x & -x`
- Builtins and `bitset` make bit counting and bulk ops fast
- `for mask in [0, 2ⁿ)` replaces subset recursion
- Submask enumeration: `s = (s - 1) & m`, total $O(3^n)$
- Solve AND/OR/XOR problems **one bit at a time**
- Next week: **number theory I** — divisibility, primes, sieves